

ALGORITHMIC TRADING WITH AI — COURSE MATERIAL

FROM TRADING IDEA TO MT5 EA WITH

AI

A Practical Step-by-Step Guide to Turning a Trading Idea Into a Working MT5 Expert Advisor

How to use ChatGPT and Claude to transform a simple trading idea
into an MT5 Expert Advisor — without writing the code yourself

PREPARED FOR LEARNERS

Worked example: Previous-Day High/Low Breakout EA (MQL5)

Edition — September 2026

CONTENTS

- 1** Introduction
- 2** The AI Trading Robot Workflow
- 3** Phase 1 — Strategy Design
- 4** Phase 2 — Coding the EA
- 5** Phase 3 — MetaEditor: Build and Compile
- 6** Phase 4 — Debugging With Claude
- 7** Phase 5 — Backtesting in Strategy Tester
- 8** Troubleshooting Guide
- 9** The Complete AI-to-EA Workflow (Reference Map)
- 10** Final Checklist
- 11** Conclusion & Quick Reference
- Appendix A** Complete MQL5 Source Code — PrevDayBreakoutEA.mq5

1 INTRODUCTION

You do not need to be an experienced programmer to begin turning your trading ideas into automated trading systems. Modern AI tools can help you move from an idea written in plain English to a working Expert Advisor (EA) for MetaTrader 5.

But there is an important distinction that every learner on this course must internalise:

KEY PRINCIPLE

AI does not replace the thinking required to design a trading strategy. You still need to explain, precisely, what you want the robot to do.

The process taught in this guide is therefore not “ask AI to make me a trading robot.” Instead, we follow a structured process:

Trading Idea !' Specification !' Review !' Coding Prompt !' MQL5 Code !' MetaEditor !' Compilation !' Backtest !' Verification

This process matters because a vague trading idea produces vague or incorrect code. The more precisely you define your trading rules, the easier it becomes for AI to build exactly what you intended.

The worked example used throughout this guide

Our example strategy is a simple Previous-Day High/Low Breakout EA. In plain terms:

- At the beginning of each trading day, identify yesterday's High and Low.
- Buy when price breaks above yesterday's High.
- Sell when price breaks below yesterday's Low.
- Use a fixed Stop Loss.
- Use a fixed Take Profit.
- Allow a configurable number of trades per direction, per day.
- Allow Buy and Sell trades independently of one another.

We will take this idea all the way through to a live backtest in the MT5 Strategy Tester.

2 THE AI TRADING ROBOT WORKFLOW

Before we begin, it is essential to understand the complete process end-to-end. There are two AI tools used in this workflow, each with a distinct role, plus two pieces of MetaTrader software.

ChatGPT — the strategy partner

We use ChatGPT primarily for:

- Developing the idea
- Asking clarifying questions
- Turning the idea into a technical specification
- Reviewing the specification
- Creating a clear coding prompt for Claude

Claude — the coding partner

We use Claude primarily for:

- Turning the specification into MQL5 code
- Producing the complete EA
- Fixing compilation errors
- Producing fresh, complete versions of the code when necessary

MetaEditor — the build environment

MetaEditor is where we create the EA file, paste in the MQL5 code, compile it, and identify any compilation errors.

MT5 Strategy Tester — the proving ground

We use the Strategy Tester to run the EA against historical data, watch how it trades, check whether it behaves according to the original strategy, and investigate problems before considering live trading.

The complete process at a glance

1. Trading Idea
2. ChatGPT
3. Strategy Specification
4. Review + Accept / Modify
5. ChatGPT
6. Claude Coding Prompt
7. Claude
8. MQL5 Code
9. MetaEditor
10. Compile
11. Fix Errors if Necessary
12. MT5 Strategy Tester
13. Backtest
14. Visual Verification

3 PHASE 1 — STRATEGY DESIGN

Step 1 — Start With Your Trading Idea

The first step is not coding — it is explaining your trading idea. Do not worry about using programming terminology; explain the strategy as you would explain it to another trader.

For our worked example, the starting idea was expressed like this:

LEARNER PROMPT EXAMPLE

"I need you to assist me create a breakout trading robot for MT5. It is based on breakout of the high/low of the previous day. It trades from the opening of the day, place a buy when it cross the High of the previous days and a sell for the low. Stop loss and take profit should be fixed number of pips."

That is enough to begin. Notice what the idea already tells us: which market levels to use, when the robot starts trading, what creates a Buy, what creates a Sell, and how Stop Loss and Take Profit work.

But important questions remain open — for example: How many trades can it take? Can it trade again after a loss? Can it trade both directions on the same day? Does it wait for candle confirmation? What exactly does "opening of the day" mean? How should lot size work? Should there be a spread filter? This is exactly why we do not immediately ask an AI to write code. First, we build the specification.

Step 2 — Ask ChatGPT to Build the Specification

Take your trading idea and give it to ChatGPT. The purpose of this step is to turn the idea into a precise, testable set of rules. A good starting prompt is:

STARTING PROMPT FOR CHATGPT

"I have a trading idea and I want to turn it into an MT5 Expert Advisor. Help me brainstorm the idea and create a complete, clear specification before we write any code. I will review the specification and decide what should be accepted, modified or removed."

Then explain your strategy. ChatGPT turns the idea into a structured specification defining things such as the trading levels used, the Buy condition, the Sell condition, Stop Loss, Take Profit, and the number of trades allowed.

Step 3 — Review and Accept the Specification

KEY PRINCIPLE

You are the strategy designer. ChatGPT is helping you translate your idea into precise rules — it is not deciding your strategy for you.

Read through every part of the specification and ask yourself: “Is this exactly how I want my trading system to behave?”

In our worked example, after review we confirmed the following decisions:

- Allow a configurable number of Buy trades per day
- Allow a configurable number of Sell trades per day
- Allow another Buy after a previous Buy loses
- Allow the opposite direction to trade after a loss
- Allow both directions to trade during the same day
- Enter immediately when price crosses the level (no candle-close wait)
- Use broker/server 00:00 as the beginning of the trading day
- Use fixed-pip Stop Loss and Take Profit
- Use a fixed lot size
- Use a spread filter

The lesson for learners: do not allow AI to silently make strategy decisions for you. If AI suggests a rule, you decide whether that rule belongs in your strategy.

Step 4 — Ask ChatGPT for the Claude Coding Prompt

Once the specification is complete and accepted, move to the coding stage. Rather than pasting an entire brainstorming conversation into Claude, ask ChatGPT to produce a clear, structured coding prompt:

REQUEST TO CHATGPT

“Then after, give me a Claude prompt.”

ChatGPT produces a detailed prompt containing the complete EA requirements. This is useful because Claude receives a clean technical specification rather than having to interpret an entire brainstorming conversation. The prompt tells Claude what the EA does, the exact Buy and Sell conditions, trade limits, SL/TP rules, lot sizing, spread filtering, daily reset behaviour, trade counting, required inputs, technical requirements — and, just as importantly, what NOT to add. This becomes the bridge between strategy design and programming.

Step 5 — Give the Prompt to Claude

Open Claude and create a new conversation. Paste the complete coding prompt from ChatGPT into Claude. You do not need to re-explain the entire strategy if the prompt already contains everything.

The prompt should explicitly ask for a complete MQL5 Expert Advisor that can be pasted into MetaEditor and compiled. Claude will then generate the MQL5 source code.

4 PHASE 2 — CODING THE EA

Step 6 — Get the MQL5 Code

Claude provides the complete MQL5 code and, depending on the environment, may also provide an actual .mq5 file. This gives you several ways to continue:

- Option 1 — Use the .mq5 file directly, if Claude provides one.
- Option 2 — Copy the complete code from Claude's reply into MetaEditor (recommended for learning the process).
- Option 3 — Keep the generated file as a backup before making any manual changes.

Appendix A of this guide contains the complete, working MQL5 source produced for our worked example — use it as a reference for what a finished, compiling EA should look like.

5 PHASE 3 — METAEDITOR: BUILD AND COMPILE

Step 7 — Put the Code Into MetaEditor

Step 1 — Open MetaTrader 5

Open MT5 normally. MetaEditor can usually be opened from inside MT5, or as a separate application.

One common method is: Tools !' MetaQuotes Language Editor. You can also use the MetaEditor button in the MT5 toolbar if available.

Step 2 — Create a new Expert Advisor

Inside MetaEditor, click File !' New !' Expert Advisor (template) !' Next. You will be asked to give the EA a name — for our example, PreviousDayBreakoutEA. Continue through the wizard to generate the EA.

Step 3 — Replace the template code

Select all of the generated template code and delete it. Copy the complete MQL5 code generated by Claude and paste it into the MetaEditor window.

Step 8 — Compile the EA

At the top of MetaEditor, click Compile (or use the equivalent shortcut). MetaEditor will process the code — look at the results panel at the bottom. We want 0 errors. Warnings may sometimes appear, but errors are what prevent compilation.

```
// MetaEditor v5.00 - [PreviousDayBreakoutEA.mq5]
#include <Trade\Trade.mqh>

input double LotSize = 0.10;      // Fixed lot size
input int    MaxBuyTradesPerDay = 3; // Max BUY trades per day
```

Errors: 0 errors, 0 warnings, elapsed time: 142 ms [Compile Success]

Figure 5.1 — MetaEditor showing a successful build: “0 errors, 0 warnings” for breakout EA.mq5.

WHAT TO LOOK FOR

A successful result reads “0 errors, 0 warnings” in the Errors panel, as shown in Figure 5.1. If you see this, the EA is ready to move on to backtesting.

6 PHASE 4 — DEBUGGING WITH CLAUDE

Step 9 — Fix Compilation Errors With Claude

Sometimes the first version of the code will not compile. This does not mean the project has failed — it means we need to debug the code, and AI becomes extremely useful here.

Step 1 — Copy the compilation errors from MetaEditor

Select and copy the full error text shown in the Errors panel.

Step 2 — Go back to Claude

Return to the same conversation where the EA was generated.

Step 3 — Paste the errors into Claude

Give Claude the exact error text, unedited.

Step 4 — Ask Claude to correct the code

Request the complete corrected file, not just the changed lines.

PROMPT TEMPLATE FOR FIXING ERRORS

"The EA does not compile in MetaEditor. Here are the compilation errors. Please fix all errors and provide the complete fresh MQL5 code. Do not provide only the changed sections. Give me the entire corrected EA."

Claude should then provide a new, complete version. Copy the fresh code, return to MetaEditor, replace the old code, and compile again. Repeat this process until the EA compiles successfully.

IMPORTANT RULE

Do not try to manually fix complicated MQL5 errors if you do not understand what they mean. Use Claude as your coding assistant and always request the entire corrected file.

7 PHASE 5 — BACKTESTING IN STRATEGY TESTER

Step 10 — Open the MT5 Strategy Tester

Once the EA compiles successfully, we are ready to test it. Compilation only tells us that the code is syntactically acceptable to the compiler — it does NOT prove that the trading strategy works correctly. That is why we need a backtest.

Open MetaTrader 5 and, from the top menu, select View !' Strategy Tester.

Step 11 — Configure the Backtest

Inside the Strategy Tester, select the EA you created (for example, breakout EA.ex5), then configure the test settings.

```
MetaTrader 5 Strategy Tester [Settings]
Expert:      PreviousDayBreakoutEA.ex5
Symbol:      GBPUSD   |   Timeframe: H1 (1 Hour)
Period:      Custom (2025.01.01 - 2026.08.31)
Execution:   Every tick based on real ticks |   Deposit: 10,000 USD
[ ' ] Visual mode with the display of charts, indicators and trades
```

Figure 7.1 — Strategy Tester Settings tab: Expert, Symbol, Timeframe, custom testing period, deposit and Visual Mode enabled.

Symbol

Choose the instrument you want to test — for example EURUSD, GBPUSD, XAUUSD, or another supported instrument. The symbol should match the market your EA is designed to trade.

Timeframe

Although our strategy uses the previous D1 candle for its breakout levels, the EA itself can operate from different chart timeframes. For the first test, choose a sensible timeframe (H1 in Figure 7.1) and remain consistent when comparing results — the EA's Previous Day High and Low always come from the completed daily candle regardless of chart timeframe.

Testing period

Choose a historical date range. For development, it is useful to test a meaningful amount of historical data rather than only a few days.

Inputs

Switch to the Inputs tab to confirm the EA's parameters match your accepted specification — lot size, max Buy/Sell trades per day, Stop Loss and Take Profit in pips, max allowed spread, and the unique magic number.

Inputs Configuration Tab

Variable	Value
LotSize	0.10
MaxBuyTradesPerDay	3
MaxSellTradesPerDay	3
StopLossPips	20.0
TakeProfitPips	40.0

Figure 7.2 — Strategy Tester Inputs tab showing the EA's configurable parameters, matching the accepted specification.

Step 12 — Use Visual Mode

For the first tests, Visual Mode is extremely useful. Why? Because we do not only want to know how much money the EA made or lost — we want to see whether the EA is actually doing what we told it to do.

Enable Visual Mode, then start the test. The historical chart will begin moving forward, and you can watch the EA operate on historical market data. This is one of the most important parts of the development process.

Visual Mode Chart Execution: GBPUSD, H1

```
=== Prev Day Breakout EA ===  
PDH: 1.28450 [Line plotted in DodgerBlue]  
PDL: 1.27800 [Line plotted in OrangeRed]  
Buy trades today: 1 / 3  
Sell trades today: 0 / 3  
Spread: 1.2 pips (max 3.0)  
%2 [BUY #1 Opened at 1.28452] SL: 1.28252 TP: 1.28852  
Status: Running
```

Figure 7.3 — Strategy Tester Visualization: the EA trading GBPUSD, H1, showing entry markers plotted against price action.

Step 13 — Check Whether the EA Behaves as Expected

Now compare what the EA does against the original specification.

Check the breakout levels

Suppose yesterday's High was 1.1050. The EA should identify that level. If price remains below it, no Buy should occur. When price crosses above it, a Buy should occur, assuming the other conditions such as spread and daily trade limits permit it. The same logic applies in reverse for yesterday's Low and Sell trades.

Check the Stop Loss

When a trade opens, check whether the Stop Loss is the correct distance from the entry. If $\text{StopLossPips} = 20$, the EA should use approximately 20 pips according to the symbol's pip convention.

Check the Take Profit

Do the same for Take Profit. If $\text{TakeProfitPips} = 40$, the Take Profit should be approximately 40 pips from the entry.

Check multiple trades

If $\text{MaxBuyTradesPerDay} = 3$, the EA should never open more than three Buy trades during that trading day. The same applies to $\text{MaxSellTradesPerDay}$ for Sell trades.

Check re-entry

This is especially important. Suppose the Previous Day High breaks and the EA opens Buy #1. The trade hits Stop Loss. Later, price breaks the same level again — the EA should be able to open Buy #2, provided the daily Buy limit has not been reached. This confirms the EA follows the intended re-entry rule.

Check opposite breakouts

Suppose the Buy breakout occurs first and then loses. Later, price breaks the Previous Day Low — the EA should still be able to take the Sell. This confirms that Buy and Sell trade limits are independent of one another.

Check the daily reset

Move through the historical test into a new trading day and confirm that the previous day's High/Low levels update, the Buy counter resets, the Sell counter resets, and the EA starts using the new previous day's levels. A mistake in daily reset logic can cause an EA to trade the wrong levels.

8 TROUBLESHOOTING GUIDE

Problem: MetaEditor shows errors

Copy the complete error information and give it to Claude. Ask for the complete corrected code (see Phase 4, Step 9).

Problem: The EA compiles but does not trade

Check the following:

- Is the market data available?
- Is the Strategy Tester configured correctly?
- Is the price actually crossing the previous day's level?
- Is the spread too high?
- Has the maximum number of daily trades already been reached?
- Is trading allowed for the symbol?
- Is the lot size valid?

Then use Visual Mode to investigate further.

Problem: The EA trades too many times

Check the cross-detection logic and daily trade counters. The EA should detect a genuine crossing rather than repeatedly interpreting every tick above the breakout level as a new breakout.

Problem: The EA uses the wrong Previous Day High/Low

Check the daily candle being used. The EA should use the completed previous D1 candle, not the current day's developing High or Low.

Problem: You find a strategy logic error

Do not simply say "the EA is wrong." Explain the exact behaviour you observed. For example:

GOOD BUG REPORT EXAMPLE

"On March 14, price broke yesterday's High, the EA opened a Buy, the Buy hit Stop Loss, and price later broke the same High again. The EA did not take the second Buy even though MaxBuyTradesPerDay was set to 3."

That gives Claude a specific, testable problem to solve — for example:

PROMPT TEMPLATE FOR LOGIC FIXES

"I backtested the EA in MT5 Visual Mode and found a logic problem. The EA opened a Buy even though price was already above the Previous Day High. I only want a Buy when price actually crosses from below the Previous Day High"

to above it. Please fix this logic and provide the complete updated MQL5 code."

Then take the new code back into MetaEditor, compile again, and backtest again. This creates a development loop: Build !' Compile !' Backtest !' Observe !' Find Problem !' Claude !' New Code !' Compile !' Backtest Again. This is normal software development.

9 THE COMPLETE AI-TO-EA WORKFLOW

You can now see the entire process laid out phase by phase. Use this as a revision map.

Phase 1 — Strategy Design

- Step 1: Write your trading idea in plain English.
- Step 2: Give the idea to ChatGPT.
- Step 3: Ask ChatGPT to create a complete technical specification.
- Step 4: Review every rule.
- Step 5: Accept, modify or remove individual rules.

Phase 2 — Coding

- Step 6: Ask ChatGPT to turn the accepted specification into a clear Claude coding prompt.
- Step 7: Copy the prompt.
- Step 8: Paste the prompt into Claude.
- Step 9: Ask Claude for the complete MQL5 EA.

Phase 3 — MetaEditor

- Step 10: Open MetaEditor.
- Step 11: Create a new Expert Advisor.
- Step 12: Give the EA a name.
- Step 13: Delete the template code.
- Step 14: Paste Claude's complete MQL5 code.
- Step 15: Compile.

Phase 4 — Debugging

- Step 16: If there are compilation errors, copy them.
- Step 17: Paste them into Claude.
- Step 18: Ask Claude for the complete corrected code.
- Step 19: Replace the code in MetaEditor.
- Step 20: Compile again — repeat until successful.

Phase 5 — Backtesting

- Step 21: Open MT5.
- Step 22: Go to View !' Strategy Tester.
- Step 23: Select your EA.
- Step 24: Select your trading symbol.
- Step 25: Select your testing period.

- Step 26: Enable Visual Mode.
- Step 27: Run the test.
- Step 28: Watch the EA trade.
- Step 29: Compare its behaviour with your original specification.
- Step 30: Document any problems.
- Step 31: Send the problem to Claude.
- Step 32: Generate the corrected code.
- Step 33: Compile again.
- Step 34: Backtest again.

10 FINAL CHECKLIST

Before considering your EA ready for further testing, work through this checklist as a class or individually.

Strategy

- ☐ Is the trading idea clearly defined?
- ☐ Are the entry conditions clear?
- ☐ Are the exit conditions clear?
- ☐ Are the Stop Loss rules clear?
- ☐ Are the Take Profit rules clear?
- ☐ Are the daily trade limits clear?

AI Specification

- ☐ Did ChatGPT turn the idea into a detailed specification?
- ☐ Did you personally review the specification?
- ☐ Did you remove rules you did not want?
- ☐ Did you add rules that were missing?
- ☐ Did you approve the final specification?

Coding

- ☐ Did you give Claude a clear coding prompt?
- ☐ Did Claude provide complete MQL5 code?
- ☐ Did you put the code into MetaEditor?
- ☐ Does the EA compile?
- ☐ Are there zero compilation errors?

Backtesting

- ☐ Did you open Strategy Tester?

- ☐ Did you select the correct EA?
- ☐ Did you select the correct symbol?
- ☐ Did you select an appropriate historical period?
- ☐ Did you enable Visual Mode?
- ☐ Did you watch the EA execute trades?
- ☐ Did the entries occur where expected?
- ☐ Did Stop Loss work correctly?
- ☐ Did Take Profit work correctly?
- ☐ Did the daily trade counter work correctly?
- ☐ Did the EA reset correctly on a new trading day?
- ☐ Did the EA allow re-entry as specified?
- ☐ Did the EA allow the opposite breakout as specified?

11 CONCLUSION & QUICK REFERENCE

The most important lesson from this process is that building an automated trading system is not simply about getting AI to write code. The real process begins with the trading idea.

You define what you want. Then you turn that idea into precise rules. Then AI helps translate those rules into software.

THE WORKFLOW, IN ONE LINE

Think !' Define !' Review !' Code !' Compile !' Test !' Observe !' Improve

ChatGPT helps you structure the strategy. Claude helps you turn the specification into MQL5. MetaEditor turns the code into a compiled MT5 Expert Advisor. The Strategy Tester allows you to see how that EA behaves against historical data. The process repeats whenever you discover something that does not behave according to your intended rules. The goal is not to create a complicated robot immediately. Start with one clearly defined idea. Build it. Compile it. Backtest it. Watch what it actually does. Then improve it. That is how you systematically turn trading ideas into automated trading systems.

The 8-Step AI EA Method

- | | |
|---------------------------|---|
| 1. IDEA | Write your trading strategy in plain English. |
| 2. CHATGPT | Ask ChatGPT to create a detailed specification. |
| 3. REVIEW | Accept, modify or remove the proposed rules. |
| 4. CLAUDE PROMPT | Ask ChatGPT to create a clear coding prompt. |
| 5. CLAUDE | Paste the prompt into Claude and receive the complete MQL5 code. |
| 6. METAEDITOR | Create an EA, paste the code and compile. |
| 7. DEBUG | If errors appear, send them back to Claude and obtain fresh code. |
| 8. STRATEGY TESTER | Backtest in MT5 using Visual Mode and verify the EA behaves according to your strategy. |

Your next EA

Once you understand this process, you can apply the same workflow to many different trading ideas, for example:

- Breakout strategies

- Moving-average strategies
- RSI strategies
- Support and resistance systems
- Session breakout systems
- Trend-following systems
- Mean-reversion systems
- Multi-indicator systems
- Automated risk-management systems
- Custom trading tools

The technology may change and the AI tools may improve, but the fundamental process remains the same: a clear trading idea, a precise specification, correctly implemented code, testing, and verification.

APPENDIX A

Complete MQL5 Source Code — PrevDayBreakoutEA.mq5

This is the complete, compiling MQL5 Expert Advisor produced for the worked example in this guide, implementing the Previous-Day High/Low Breakout strategy exactly as specified in Chapter 3. Learners can paste this directly into MetaEditor to reproduce Figure 5.1 (a clean, zero-error compile), or use it as a reference point when comparing their own AI-generated code.

HOW TO USE THIS APPENDIX

Copy the code below into a new Expert Advisor file in MetaEditor, replacing the template. Compile, then follow Phase 5 of this guide to backtest it in Strategy Tester.

```
//+-----+
//|                                     PrevDayBreakoutEA.mq5 |
//|               Previous Day High/Low Breakout Expert Advisor |
//|
//| Strategy:
//| - At the start of each broker trading day, read the completed
//|   previous D1 candle and store its High/Low as fixed breakout
//|   levels for the current day.
//| - Open a BUY the instant price genuinely crosses above the
//|   Previous Day High (not merely "is above" it).
//| - Open a SELL the instant price genuinely crosses below the
//|   Previous Day Low (not merely "is below" it).
//| - Independent, configurable max trades per direction per day.
//| - Fixed pip SL/TP, fixed lot size, max spread filter.
//| - Counters are reconstructed from trade history on restart.
//+-----+
#property copyright "Generated EA"
#property version   "1.00"
#property strict

#include <Trade\Trade.mqh>

//=====
// INPUTS
//=====
input double LotSize           = 0.10;    // Fixed lot size
input int    MaxBuyTradesPerDay = 3;       // Max BUY trades per day
input int    MaxSellTradesPerDay = 3;      // Max SELL trades per day
input double StopLossPips      = 20;      // Stop Loss in pips
input double TakeProfitPips    = 40;      // Take Profit in pips
input double MaxSpreadPips     = 3.0;     // Max allowed spread in pips
input long   MagicNumber       = 20260917; // Unique magic number

//=====
// GLOBALS
//=====
CTrade g_trade;

double g_pipSize      = 0.0; // value of 1 pip in price terms
double g_point        = 0.0;
int     g_digits       = 0;

datetime g_currentDayStart = 0; // 00:00 of the broker day currently loaded
double   g_prevDayHigh    = 0.0;
double   g_prevDayLow     = 0.0;
```

```
int      g_buyCountToday    = 0;
int      g_sellCountToday   = 0;

double   g_prevBid          = 0.0;
double   g_prevAsk          = 0.0;
bool     g_priceBaselineSet = false;

string   g_objPrefix        = ""; // unique per magic number, used for all chart objects

string   g_statusText       = "Initializing";

//=====
// UTILITY: pip size calculation (handles 3/5 digit and 2/4 digit)
//=====
double CalcPipSize()
{
    int digits = (int)SymbolInfoInteger(_Symbol, SYMBOL_DIGITS);
    double point = SymbolInfoDouble(_Symbol, SYMBOL_POINT);

    if(digits == 3 || digits == 5)
        return point * 10.0;

    return point;
}

//=====
// UTILITY: broker-day start (00:00 server time) for a given time
//=====
datetime DayStart(datetime t)
{
    MqlDateTime mt;
    TimeToStruct(t, mt);
    mt.hour = 0;
    mt.min  = 0;
    mt.sec  = 0;
    return StructToTime(mt);
}

//=====
// UTILITY: validate/normalize lot size against symbol constraints
//=====
double ValidateLot(double lots)
{
    double minLot = SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_MIN);
    double maxLot = SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_MAX);
    double lotStep = SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_STEP);

    if(lotStep <= 0.0)
        lotStep = 0.01;

    double steps = MathRound(lots / lotStep);
    double normalized = steps * lotStep;

    if(normalized < minLot) normalized = minLot;
    if(normalized > maxLot) normalized = maxLot;

    // Guard against floating point noise
    int lotDigits = 2;
    double stepCheck = lotStep;
    while(MathAbs(stepCheck - MathRound(stepCheck)) > 0.0000001 && lotDigits < 6)
    {
        stepCheck *= 10.0;
    }
}
```

```
        lotDigits++;
    }
    normalized = NormalizeDouble(normalized, lotDigits);

    return normalized;
}

//=====
// UTILITY: current spread in pips
//=====
double GetSpreadPips()
{
    double bid = SymbolInfoDouble(_Symbol, SYMBOL_BID);
    double ask = SymbolInfoDouble(_Symbol, SYMBOL_ASK);

    if(g_pipSize <= 0.0)
        return 0.0;

    return (ask - bid) / g_pipSize;
}

//=====
// Reconstruct today's Buy/Sell counters from trade history
// (so a restart mid-day does not lose or duplicate the count)
//=====
void RecalculateDailyCounts()
{
    g_buyCountToday = 0;
    g_sellCountToday = 0;

    datetime fromTime = g_currentDayStart;
    datetime toTime = TimeTradeServer() + 60; // small buffer

    if(!HistorySelect(fromTime, toTime))
    {
        Print("PrevDayBreakoutEA: HistorySelect failed, error=", GetLastError());
        return;
    }

    int total = HistoryDealsTotal();
    for(int i = 0; i < total; i++)
    {
        ulong dealTicket = HistoryDealGetTicket(i);
        if(dealTicket == 0)
            continue;

        long dealMagic = HistoryDealGetInteger(dealTicket, DEAL_MAGIC);
        string dealSymbol = HistoryDealGetString(dealTicket, DEAL_SYMBOL);
        long dealEntry = HistoryDealGetInteger(dealTicket, DEAL_ENTRY);
        long dealTime = HistoryDealGetInteger(dealTicket, DEAL_TIME);

        if(dealMagic != MagicNumber)
            continue;
        if(dealSymbol != _Symbol)
            continue;
        if(dealEntry != DEAL_ENTRY_IN)
            continue;
        if((datetime)dealTime < g_currentDayStart)
            continue;

        long dealType = HistoryDealGetInteger(dealTicket, DEAL_TYPE);
        if(dealType == DEAL_TYPE_BUY)
            g_buyCountToday++;
        else if(dealType == DEAL_TYPE_SELL)
```

```
        g_sellCountToday++;
    }

    Print("PrevDayBreakoutEA: Reconstructed daily counts - Buy=", g_buyCountToday,
        " Sell=", g_sellCountToday);
}

//=====
// Load the completed previous D1 candle's High/Low
//=====
bool LoadPreviousDayLevels()
{
    double highs[];
    double lows[];
    ArraySetAsSeries(highs, true);
    ArraySetAsSeries(lows, true);

    // Make sure there is enough D1 history
    if(Bars(_Symbol, PERIOD_D1) < 2)
    {
        Print("PrevDayBreakoutEA: Not enough D1 history yet.");
        return false;
    }

    int copiedH = CopyHigh(_Symbol, PERIOD_D1, 1, 1, highs);
    int copiedL = CopyLow(_Symbol, PERIOD_D1, 1, 1, lows);

    if(copiedH <= 0 || copiedL <= 0)
    {
        Print("PrevDayBreakoutEA: Failed to copy previous D1 high/low, error=", GetLastError());
        return false;
    }

    g_prevDayHigh = highs[0];
    g_prevDayLow = lows[0];

    Print("PrevDayBreakoutEA: Loaded PrevDayHigh=", DoubleToString(g_prevDayHigh, g_digits),
        " PrevDayLow=", DoubleToString(g_prevDayLow, g_digits));
    return true;
}

//=====
// Handle detection of a new broker trading day + full daily reset
//=====
void CheckNewDay()
{
    datetime nowServer = TimeTradeServer();
    datetime todayStart = DayStart(nowServer);

    if(todayStart == g_currentDayStart && g_currentDayStart != 0)
        return; // still the same day, nothing to do

    g_currentDayStart = todayStart;

    if(!LoadPreviousDayLevels())
    {
        g_statusText = "Waiting for D1 data";
        return;
    }

    RecalculateDailyCounts();

    // Re-arm the crossing baseline: the next tick will just record price,
```

```
// not trade, so we never fire a false breakout right after a reset.
g_priceBaselineSet = false;

DrawLevelLines();
g_statusText = "Running";
Print("PrevDayBreakoutEA: New trading day detected. Levels and counters reset.");
}

//=====
// Chart objects: horizontal lines for PDH / PDL
//=====
void DrawLevelLines()
{
    string highName = g_objPrefix + "PDH_Line";
    string lowName = g_objPrefix + "PDL_Line";

    if(ObjectFind(0, highName) < 0)
    {
        ObjectCreate(0, highName, OBJ_HLINE, 0, 0, g_prevDayHigh);
        ObjectSetInteger(0, highName, OBJPROP_COLOR, clrDodgerBlue);
        ObjectSetInteger(0, highName, OBJPROP_STYLE, STYLE_DASH);
        ObjectSetInteger(0, highName, OBJPROP_WIDTH, 1);
        ObjectSetInteger(0, highName, OBJPROP_BACK, true);
        ObjectSetInteger(0, highName, OBJPROP_SELECTABLE, false);
        ObjectSetString(0, highName, OBJPROP_TEXT, "Previous Day High");
    }
    else
    {
        ObjectSetDouble(0, highName, OBJPROP_PRICE, g_prevDayHigh);
    }

    if(ObjectFind(0, lowName) < 0)
    {
        ObjectCreate(0, lowName, OBJ_HLINE, 0, 0, g_prevDayLow);
        ObjectSetInteger(0, lowName, OBJPROP_COLOR, clrOrangeRed);
        ObjectSetInteger(0, lowName, OBJPROP_STYLE, STYLE_DASH);
        ObjectSetInteger(0, lowName, OBJPROP_WIDTH, 1);
        ObjectSetInteger(0, lowName, OBJPROP_BACK, true);
        ObjectSetInteger(0, lowName, OBJPROP_SELECTABLE, false);
        ObjectSetString(0, lowName, OBJPROP_TEXT, "Previous Day Low");
    }
    else
    {
        ObjectSetDouble(0, lowName, OBJPROP_PRICE, g_prevDayLow);
    }
}

//=====
// Chart objects: dashboard labels
//=====
void CreateLabel(string name, int yOffset)
{
    if(ObjectFind(0, name) >= 0)
        return;

    ObjectCreate(0, name, OBJ_LABEL, 0, 0, 0);
    ObjectSetInteger(0, name, OBJPROP_CORNER, CORNER_LEFT_UPPER);
    ObjectSetInteger(0, name, OBJPROP_XDISTANCE, 10);
    ObjectSetInteger(0, name, OBJPROP_YDISTANCE, yOffset);
    ObjectSetInteger(0, name, OBJPROP_COLOR, clrWhite);
    ObjectSetInteger(0, name, OBJPROP_FONTSIZE, 9);
    ObjectSetString(0, name, OBJPROP_FONT, "Consolas");
    ObjectSetInteger(0, name, OBJPROP_SELECTABLE, false);
    ObjectSetInteger(0, name, OBJPROP_HIDDEN, true);
}
}
```

```
void CreateDashboard()
{
    CreateLabel(g_objPrefix + "lbl_title", 15);
    CreateLabel(g_objPrefix + "lbl_pdh", 32);
    CreateLabel(g_objPrefix + "lbl_pdl", 49);
    CreateLabel(g_objPrefix + "lbl_buy", 66);
    CreateLabel(g_objPrefix + "lbl_sell", 83);
    CreateLabel(g_objPrefix + "lbl_spread", 100);
    CreateLabel(g_objPrefix + "lbl_status", 117);

    ObjectSetString(0, g_objPrefix + "lbl_title", OBJPROP_TEXT,
        "=== Prev Day Breakout EA ===");
    ObjectSetInteger(0, g_objPrefix + "lbl_title", OBJPROP_COLOR, clrYellow);
}

void UpdateDashboard()
{
    ObjectSetString(0, g_objPrefix + "lbl_pdh", OBJPROP_TEXT,
        "PDH: " + DoubleToString(g_prevDayHigh, g_digits));
    ObjectSetString(0, g_objPrefix + "lbl_pdl", OBJPROP_TEXT,
        "PDL: " + DoubleToString(g_prevDayLow, g_digits));

    ObjectSetString(0, g_objPrefix + "lbl_buy", OBJPROP_TEXT,
        "Buy trades today: " + IntegerToString(g_buyCountToday) +
        " / " + IntegerToString(MaxBuyTradesPerDay));

    ObjectSetString(0, g_objPrefix + "lbl_sell", OBJPROP_TEXT,
        "Sell trades today: " + IntegerToString(g_sellCountToday) +
        " / " + IntegerToString(MaxSellTradesPerDay));

    double spread = GetSpreadPips();
    color spreadColor = (spread > MaxSpreadPips) ? clrRed : clrLimeGreen;
    ObjectSetString(0, g_objPrefix + "lbl_spread", OBJPROP_TEXT,
        "Spread: " + DoubleToString(spread, 1) +
        " pips (max " + DoubleToString(MaxSpreadPips, 1) + ")");
    ObjectSetInteger(0, g_objPrefix + "lbl_spread", OBJPROP_COLOR, spreadColor);

    ObjectSetString(0, g_objPrefix + "lbl_status", OBJPROP_TEXT,
        "Status: " + g_statusText);
}

void RemoveDashboard()
{
    ObjectDelete(0, g_objPrefix + "lbl_title");
    ObjectDelete(0, g_objPrefix + "lbl_pdh");
    ObjectDelete(0, g_objPrefix + "lbl_pdl");
    ObjectDelete(0, g_objPrefix + "lbl_buy");
    ObjectDelete(0, g_objPrefix + "lbl_sell");
    ObjectDelete(0, g_objPrefix + "lbl_spread");
    ObjectDelete(0, g_objPrefix + "lbl_status");
    ObjectDelete(0, g_objPrefix + "PDH_Line");
    ObjectDelete(0, g_objPrefix + "PDL_Line");
}

//=====
// Trade execution helpers
//=====
void OpenBuyTrade(double entryPrice)
{
    double lots = ValidateLot(LotSize);
    double sl = 0.0, tp = 0.0;

    if(StopLossPips > 0)
        sl = NormalizeDouble(entryPrice - StopLossPips * g_pipSize, g_digits);
    if(TakeProfitPips > 0)
```

```

        tp = NormalizeDouble(entryPrice + TakeProfitPips * g_pipSize, g_digits);

g_trade.SetExpertMagicNumber(MagicNumber);
bool ok = g_trade.Buy(lots, _Symbol, 0.0, sl, tp, "PrevDayBreakout-Buy");
if(!ok)
{
    Print("PrevDayBreakoutEA: BUY order failed. Retcode=", g_trade.ResultRetcode(),
        " (" , g_trade.ResultRetcodeDescription(), ")");
    return;
}

uint retcode = g_trade.ResultRetcode();
if(retcode == TRADE_RETCODE_DONE || retcode == TRADE_RETCODE_DONE_PARTIAL)
{
    g_buyCountToday++;
    Print("PrevDayBreakoutEA: BUY opened. Count today=", g_buyCountToday,
        " Price=", DoubleToString(entryPrice, g_digits));
}
else
{
    Print("PrevDayBreakoutEA: BUY order not confirmed done. Retcode=", retcode,
        " (" , g_trade.ResultRetcodeDescription(), ")");
}
}

void OpenSellTrade(double entryPrice)
{
    double lots = ValidateLot(LotSize);
    double sl = 0.0, tp = 0.0;

    if(StopLossPips > 0)
        sl = NormalizeDouble(entryPrice + StopLossPips * g_pipSize, g_digits);
    if(TakeProfitPips > 0)
        tp = NormalizeDouble(entryPrice - TakeProfitPips * g_pipSize, g_digits);

    g_trade.SetExpertMagicNumber(MagicNumber);
    bool ok = g_trade.Sell(lots, _Symbol, 0.0, sl, tp, "PrevDayBreakout-Sell");
    if(!ok)
    {
        Print("PrevDayBreakoutEA: SELL order failed. Retcode=", g_trade.ResultRetcode(),
            " (" , g_trade.ResultRetcodeDescription(), ")");
        return;
    }

    uint retcode = g_trade.ResultRetcode();
    if(retcode == TRADE_RETCODE_DONE || retcode == TRADE_RETCODE_DONE_PARTIAL)
    {
        g_sellCountToday++;
        Print("PrevDayBreakoutEA: SELL opened. Count today=", g_sellCountToday,
            " Price=", DoubleToString(entryPrice, g_digits));
    }
    else
    {
        Print("PrevDayBreakoutEA: SELL order not confirmed done. Retcode=", retcode,
            " (" , g_trade.ResultRetcodeDescription(), ")");
    }
}

//=====
// Core breakout / crossing detection, runs every tick
//=====
void CheckBreakouts()

```

```
{
    if(g_prevDayHigh <= 0.0 || g_prevDayLow <= 0.0)
        return; // levels not ready yet

    double bid = SymbolInfoDouble(_Symbol, SYMBOL_BID);
    double ask = SymbolInfoDouble(_Symbol, SYMBOL_ASK);

    if(bid <= 0.0 || ask <= 0.0)
        return;

    // First tick after init / new day: just record the baseline, do not trade,
    // so we never fire a false "breakout" purely because of where price
    // already happens to be.
    if(!g_priceBaselineSet)
    {
        g_prevBid = bid;
        g_prevAsk = ask;
        g_priceBaselineSet = true;
        return;
    }

    double spreadPips = GetSpreadPips();
    bool spreadOk = (spreadPips <= MaxSpreadPips);

    // --- BUY breakout: genuine upward cross of Previous Day High ---
    bool buyCross = (g_prevAsk <= g_prevDayHigh && ask > g_prevDayHigh);
    if(buyCross && g_buyCountToday < MaxBuyTradesPerDay)
    {
        if(spreadOk)
            OpenBuyTrade(ask);
        else
            Print("PrevDayBreakoutEA: BUY breakout detected but spread too high (",
                DoubleToString(spreadPips, 1), " pips). Skipped.");
    }

    // --- SELL breakout: genuine downward cross of Previous Day Low ---
    bool sellCross = (g_prevBid >= g_prevDayLow && bid < g_prevDayLow);
    if(sellCross && g_sellCountToday < MaxSellTradesPerDay)
    {
        if(spreadOk)
            OpenSellTrade(bid);
        else
            Print("PrevDayBreakoutEA: SELL breakout detected but spread too high (",
                DoubleToString(spreadPips, 1), " pips). Skipped.");
    }

    // Update baseline for next tick - this is what prevents repeated
    // trades from a single breakout event (once "current" == "previous side",
    // no further cross is detected until price returns and crosses again).
    g_prevBid = bid;
    g_prevAsk = ask;
}

//=====
// EXPERT EVENT HANDLERS
//=====
int OnInit()
{
    g_digits = (int)SymbolInfoInteger(_Symbol, SYMBOL_DIGITS);
    g_point = SymbolInfoDouble(_Symbol, SYMBOL_POINT);
    g_pipSize = CalcPipSize();

    g_objPrefix = "PDBO_" + IntegerToString(MagicNumber) + "_";
```

```
g_trade.SetExpertMagicNumber(MagicNumber);
g_trade.SetDeviationInPoints(10);
g_trade.SetTypeFilling(ORDER_FILLING_FOK);

if(LotSize <= 0.0)
{
    Print("PrevDayBreakoutEA: LotSize must be > 0.");
    return(INIT_PARAMETERS_INCORRECT);
}
if(MaxBuyTradesPerDay < 0 || MaxSellTradesPerDay < 0)
{
    Print("PrevDayBreakoutEA: MaxBuyTradesPerDay/MaxSellTradesPerDay must be >= 0.");
    return(INIT_PARAMETERS_INCORRECT);
}

g_currentDayStart = 0; // force a fresh daily-reset on first CheckNewDay()
g_priceBaselineSet = false;

CreateDashboard();
CheckNewDay(); // load levels + reconstruct counters immediately at init
UpdateDashboard();

EventSetTimer(1);
return(INIT_SUCCEEDED);
}

void OnDeinit(const int reason)
{
    EventKillTimer();
    RemoveDashboard();
}

void OnTick()
{
    CheckNewDay();
    CheckBreakouts();
    UpdateDashboard();
}

void OnTimer()
{
    UpdateDashboard();
}
//+-----+
```